МИРЭА
Российский технологический университет

Язык Си и динамическое выделение памяти

Мухортова Н.Н.

Цель

Освоить работу с текстовыми и бинарными файлами: запись в файл и чтение из файла.

Изучить понятия, классификацию, объявления и особенности доступа к данным в динамических структурах, работу с памятью при использовании структур в программе, научиться решать задачи с использованием динамических структур в языке С.

Файлы

Открытие и закрытие потоков

Чтобы работать с потоком, его необходимо открыть. Для открытия потока применяется функция **fopen()**, которая имеет следующий прототип:

```
1   FILE * fopen(имя_файла, режим_открытия);
```

Первый параметр представляет имя открываемого файла, а второй задает режим открытия, от которого зависит, как файл может быть обработан.

Функция возвращает указатель на структуру, которая имеет тип **FILE**, определенный в том же файле stdio.h. Этот указатель идентифицирует поток в программе и через него мы сможем обращаться к открытому файлу.

Режимы открытия

- **"w"**: текстовый файл открывается для записи. Если файл ранее существовал, то он пересоздается и записывается заново
- **"r"**: текстовый файл открывается для чтения
- **"a"**: текстовый файл открывается для добавления в него новых данных. Если файл существовал ранее, то данные просто добавляются

- **"w+"**: текстовый файл создается для записи/записи. Если файл ранее существовал, то при первой записи после открытия он пересоздается и записывается заново. А при последующих записях после открытия данные добавляются в него без перезаписи.
- **"r+"**: текстовый файл открывается для чтения/записи. Запись допустима в любом месте файла, кроме конца файла, то есть недопустимо увеличение размеров файла.
- **"a+"**: текстовый файл открывается или создается (при его отсутствии) для чтения/записи. В отличие от режима w+ файл при открытии не пересоздается заново, а в отличии от режима r+ можно записывать данные в конец файла

- **"wb"**: бинарный файл открывается для записи
- **"rb"**: бинарный файл открывается для чтения
- **"ab"**: бинарный файл открывается для дозаписи
- **"w+b"**: бинарный файл создается для записи/чтения
- **"r+b"**: бинарный файл открывается для чтения/записи
- **"a+b"**: бинарный файл открывается или создается (при его отсутствии) для чтения/дозаписи

Режимы позволяют разграничить доступ к файлу и открыть его только для чтения или только для записи или совместить оба варианта. Кроме того, на уровне режимов происходит разделение файлов на текстовые и бинарные. И программа будет обрабатывать файлы определенным образом, в зависимости какой режим будет выбран - для текстовых или бинарных файлов. Неправильно заданный режим может привести к некорректной интерпретации файла.

Закрытие файла

После завершения работы с файлом его следует закрыть. Для этого применяется функция **fclose()**:

```
1   int fclose(указатель_на_поток);
```

Единственный параметр функции представляет ранее полученный при открытии файла указатель на структуру FILE, связанный с файлом.

Закрытие файла

Функция возвращает число: 0 - в случае успешного выполнения и встроенное значение **EOF** в случае ошибки. Например, откроем и закроем файл "data.txt", который расположен в одной и той же папке, что и выполняемая программа:

```c
#include <stdio.h>

int main(void)
{
    FILE * fp = fopen("data.txt", "w");
    fclose(fp);
    return 0;
}
```

Если файл расположен в каком-то другом месте, то можно указать соответствующий относительный или абсолютный путь. Например, "C:\\data.txt" (если файл `data.txt` расположен в корне диска C).

Обработка ошибок

В процессе открытия или создания файла мы можем столкнуться с рядом ошибок, например, при открытии в режиме чтения не окажется подобного файла, недостаточно памяти и т.д. И в случае возникновения ошибки функция `fopen()` возвращает значение **NULL**. Мы можем обработать возникновение ошибки с помощью проверки результата функции:

```
1
2
3    #include <stdio.h>
4
5    int main(void)
6    {
7        FILE * fp= fopen("data28.txt", "r");
8        if(fp==NULL)
9        {
1            perror("Error occured while opening data28.txt");
0            return 1;
1        }
1
1        fclose(fp);
2        return 0;
1    }
3
1
4
```

Запись файла

```
int main(void)
{
    // записываемая строка
    char message[] = "Hello METANIT.COM!";
    // файл для записи
    char * filename = "data.txt";
    // находим длину записываемой строки
    int n = sizeof(message)/sizeof(message[0]);

    FILE * fp= fopen(filename, "w");
    if(fp)
    {
        // посимвольно записываем в файл
        for(int i=0; i < n; i++)
        {
            putc(message[i], fp);
        }

        fclose(fp);
        printf("File has been written\n");
    }
}
```

Чтение файла

```c
#include <stdio.h>

int main(void)
{
    // символ для считывания
    char c;
    // файл для чтения
    char * filename = "data.txt";

    FILE * fp= fopen(filename, "r");
    if(fp)
    {
        // считываем посимвольно из файла
        while((c=getc(fp))!= EOF)
        {
            printf("%c", c);
        }
        fclose(fp);
    }
}
```

fgets – чтение строки из указанного потока

```c
int main(void)
{
    // файл чтения
    char * filename = "data.txt";
    // буфер для считавания данных из файла
    char buffer[256];
    // чтение из файла
    FILE *fp = fopen(filename, "r");
    if(fp)
    {
        // пока не дойдем до конца, считываем по 256 байт
        while((fgets(buffer, 256, fp))!=NULL)
        {
            printf("%s", buffer);
        }
        fclose(fp);
    }
}
```

fgets – запись строки в файл

```c
int main(void)
{
    char * filename1 = "data1.txt";
    char * filename2 = "data2.txt";
    char buffer[256];
    FILE *f1 = fopen(filename1, "r");   // файл на чтение
    FILE *f2 = fopen(filename2, "w");   // файл на запись
    if(!f1 || !f2)
    {
        printf("Error occured while opening file\n");
    }
    else
    {
        // пока не дойдем до конца, считываем по 256 байт из файла f1
        while((fgets(buffer, 256, f1))!=NULL)
        {
            // записываем строку в файл f2
            fputs(buffer, f2);
            printf("%s", buffer);
        }
    }

    fclose(f1);
    fclose(f2);

    return 0;
}
```

Чтение и запись структур в файл

Описание структур

```c
#include <stdio.h>
#include <stdlib.h>

struct person
{
    char name[16];
    int age;
};

int save(char * filename, struct person *p);
int load(char * filename);

int main(void)
{
    char * filename = "person.dat";
    struct person tom = { "Tom Smith", 21 };

    save(filename, &tom);
    load(filename);

    return 0;
}
```

Пример использования функции **getc()/putc()** при работе со структурами

Запись структуры в файл

```
int save(char * filename, struct person *p)
{
    FILE * fp;
    char *c;
    int size = sizeof(struct person); // количество записываемых байтов

    fp = fopen(filename, "wb"); //открываем файл для бинарной записи
    if (!fp)     // если не удалось открыть файл
    {
        printf("Error occured while opening file \n");
        return 1;
    }
    // устанавливаем указатель на начало структуры
    c = (char *)p;
    // посимвольно записываем в файл структуру
    for (int i = 0; i < size; i++)
    {
        putc(*c++, fp);
    }
    fclose(fp);
    return 0;
}
```

Чтение структуры из файла

```
int load(char * filename)
{
    FILE * fp;
    char *c;
    int i; // для считывания одного символа
    // количество считываемых байтов
    int size = sizeof(struct person);
    // выделяем память для считываемой структуры
    struct person * ptr = malloc(size);
    fp = fopen(filename, "rb");   // открываем файл для бинарного чтения
    if (!fp)
    {
        printf("Error occured while opening file \n");
        return 1;
    }
```

Чтение структуры из файла

```
// устанавливаем указатель на начало блока выделенной памяти
    c = (char *)ptr;
    // считываем посимвольно из файла
    while ((i = getc(fp))!=EOF)
    {
        *c = i;
        c++;
    }

    fclose(fp);
    // вывод на консоль загруженной структуры
    printf("%-20s %5d \n", ptr->name, ptr->age);
    free(ptr);
    return 0;
}
```

Чтение и запись массива структур в файл

Описание массива структур

```c
#include <stdio.h>
#include <stdlib.h>

struct person
{
    char name[20];
    int age;
};

int save(char * filename, struct person *st, int n);
int load(char * filename);

int main(void)
{
    char * filename = "people.dat";
    struct person people[] = { {"Tom", 23}, {"Alice", 27},
{"Bob", 31}, {"Kate", 29 }};
    int n = sizeof(people) / sizeof(people[0]);

    save(filename, people, n);
    load(filename);
    return 0;
}
```

Запись в файл массива структур

```
int save(char * filename, struct person * st, int n)
{
    char *c;   // для записи отдельных символов
    // число записываемых байтов
    int size = n * sizeof(struct person);
    FILE * fp = fopen(filename, "wb");
    if (!fp)
    {
        printf("Error occured while opening file\n");
        return -1;
    }
    // записываем количество структур
    c = (char *)&n;
    for (int i = 0; i < sizeof(n); i++)
    {
        putc(*c++, fp);
    }

    // посимвольно записываем в файл все структуры
    c = (char *)st;
    for (int i = 0; i < size; i++)
    {
        putc(*c, fp);
        c++;
    }
    fclose(fp);
    return 0;
}
```

Чтение из файла массива структур

```
int load(char * filename)
{
    char *c; // для считывания отдельных символов
    int m = sizeof(int);  // сколько надо считать для получения размера
массива
    int n;  // количество структур в массиве

    FILE * fp = fopen(filename, "r");
    if (!fp)
    {
        printf("Error occured while opening file\n");
        return -1;
    }
    // выделяем память для хранения количества данных
    int *ptr_count = malloc(m);
    // считываем количество структур
    c = (char *)ptr_count;
    // пока не считаем m байт, сохраняем байт в выделенный блок для
размера массива
    while (m > 0 && (*c = getc(fp)) != EOF)
    {
        c++;
        m--;
    }
```

Чтение из файла массива структур

```
    //получаем число элементов
    n = *ptr_count;
    free(ptr_count); // освобождаем память
    // выделяем память для считанного массива структур
    struct person * ptr = malloc(n * sizeof(struct person));
    // устанавливаем указатель на блок памяти, выделенный для массива структур
    c = (char *)ptr;
    // считываем посимвольно из файла
    while ((*c= getc(fp))!=EOF)
    {
        c++;
    }
    // перебор загруженных элементов и вывод на консоль
    printf("\nFound %d people\n\n", n);

    for (int i = 0; i < n; i++)
    {
        printf("%-5d %-10s %5d \n", i + 1, (ptr + i)->name, (ptr + i)->age);
        // или так
        // printf("%-5d %-10s %5d \n", i + 1, ptr[i].name, ptr[i].age);
    }

    free(ptr);
    fclose(fp);
    return 0;
}
```

Запись в файл с помощью функции fwrite

```
size_t fwrite( const void *buffer, size_t size, size_t count, FILE *stream );
```

Эта функция принимает следующие параметры:

- **buffer**: указатель на первый объект из массива, который должен быть записан
- **size**: размер **каждого** объекта
- **count**: количество объектов, которые надо записать
- **stream** указатель на файловый поток для записи

В качестве результата функция возвращает количество записанных объектов. Это количество может быть меньше параметра `count`, если при записи произогла ошибка.

Чтение из файла с помощью функции fread

```
size_t fread( const void *buffer, size_t size, size_t count, FILE *stream );
```

Эта функция принимает следующие параметры:

- **buffer**: указатель на массив, в который надо считать данные
- **size**: размер **каждого** объекта
- **count**: количество объектов, которые надо считать
- **stream** указатель на файловый поток для чтения

В качестве результата функция возвращает количество считанных объектов. Это количество может быть меньше параметра `count`, если при записи произогла ошибка или достигнут конец файла.

Указатели

Определение указателя

Для определения указателя надо указать тип объекта, на который указывает указатель, и символ звездочки *.

```
тип_данных* название_указателя;
```

Сначала идет тип данных, на который указывает указатель, и символ звездочки *. Затем имя указателя.

```
int *p; // указатель на тип int
```

Определение указателя

Теперь присвоим ему адрес переменной:

```
int main(void)
{
    int x = 10;     // определяем переменную
    int *p;         // определяем указатель
    p = &x;         // указатель получает адрес переменной
    return 0;
}
```

Получение адреса данных

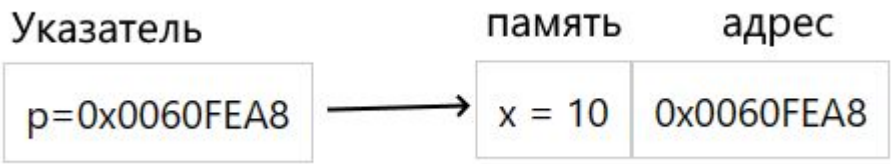
Указатель хранит адрес объекта в памяти компьютера. И для получения адреса к переменной применяется операция **&**. Эта операция применяется только к таким объектам, которые хранятся в памяти компьютера, то есть к переменным и элементам массива.

Что важно, переменная x имеет тип int, и указатель, который указывает на ее адрес тоже имеет тип int. То есть должно быть соответствие по типу.

Какой именно адрес имеет переменная x? Для вывода значения указателя можно использовать специальный спецификатор **%p**

В данном случае машинный адрес переменной x - 0x0060FEA8. (Для адресов в памяти применяется шестнадцатеричная система.) Но в каждом отдельном случае адрес может быть иным. Фактически адрес представляет целочисленное значение, выраженное в шестнадцатеричном формате.

То есть в памяти компьютера есть адрес 0x0060FEA8, по которому располагается переменная x.



Так как переменная x представляет тип **int**, то на большинстве архитектур она будет занимать следующие 4 байта (на конкретных архитектурах размер памяти для типа int может отличаться). Таким образом, переменная типа int последовательно займет ячейки памяти с адресами 0x0060FEA8, 0x0060FEA9, 0x0060FEAA, 0x0060FEAB.

объект типа int занимает 4 байта

60FEA8	60FEA9	60FEAA	60FEAB
1 байт	1 байт	1 байт	1 байт
10			
переменная x			

И указатель **p** будет ссылаться на адрес, по которому располагается переменная x, то есть на адрес 0x0060FEA8.

Получение значения по адресу

Но так как указатель хранит адрес, то мы можем по этому адресу получить хранящееся там значение, то есть значение переменной x. Для этого применяется операция * или операция разыменования (dereference operator). Результатом этой операции всегда является объект, на который указывает указатель. Применим данную операцию и получим значение переменной x:

Получение значения по адресу

```c
#include <stdio.h>

int main(void)
{
    int x = 10;
    int *p;
    p = &x;
    printf("Address = %p \n",
(void*) p);
    printf("x = %d \n", *p);
    return 0;
}
```

Консольный вывод:

```
Address = 0060FEA8
x = 10
```

```c
#include <stdio.h>

int main(void)
{
    char c = 'N';
    int d = 10;
    short s = 2;

    char *pc = &c;          // получаем адрес переменной c типа char
    int *pd = &d;           // получаем адрес переменной d типа int
    short *ps = &s;         // получаем адрес переменной s типа short

    printf("Variable c: address=%p \t value=%c \n", (void*) pc, *pc);
    printf("Variable d: address=%p \t value=%d \n", (void*) pd, *pd);
    printf("Variable s: address=%p \t value=%hd \n", (void*) ps, *ps);
    return 0;
}
```

Операции с указателями

Указателю можно присвоить либо адрес объекта того же типа, либо значение другого указателя или константу **NULL**.

Присвоение указателю адреса уже рассматривалось в прошлой теме. Для получения адреса объекта используется операция **&**:

Присваивание

```
int a = 10;
int *pa = &a;   // указатель pa хранит адрес переменной a
```

```
int *pa = NULL;
```

```c
#include <stdio.h>

int main(void)
{
    int a = 10;

    int *pa = &a;
    int *pb = pa;

    *pa = 25;

    printf("Value on pointer pa: %d \n", *pa);  // 25
    printf("Value on pointer pb: %d \n", *pb);  // 25
    printf("Value of variable a: %d \n", a);    // 25

    return 0;
}
```

Разыменование указателя

Операция разыменования указателя в виде `*имя_указателя`, позволяет получить объект по адресу, который хранится в указателе.

Управление памятью

В этой лекции мы рассмотрим все возможные способы выделения памяти в различных областях программы, разберем достоинства и недостатки всех методов. Мы также изучим некоторые способы установки содержимого произвольно выбранных областей памяти и оперирования этим содержимым. Мы научимся блокировать некоторый объем памяти, чтобы он не покидал пределов RAM (оперативной памяти) и ваша программа продолжала работать. При этом программе не придется дожидаться, пока ядро скопирует данные из области подкачки.

Память - это один из простейших и в то же время фундаментальных ресурсов, доступных для использования процессами. В этой лекции мы поговорим об управлении памятью - как ее выделять, манипулировать ею, а затем высвобождать.

Термин “выделять”, который обычно обозначает получение памяти, немного неточен. Он ассоциируется с распределением скудного ресурса, спрос на который превышает предложение. Честно говоря, многие пользователи действительно желают получить как можно больше памяти. В современных системах основные сложности работы с памятью связаны не с тем, чтобы удовлетворить потребности всех нуждающихся, располагая небольшим объемом памяти, а с тем, чтобы правильно использовать выделенную дозу памяти и отслеживать процесс такого использования.

Адресное пространство процесса

Linux, как и все современные операционные системы, виртуализирует имеющуюся в распоряжении физическую память. Процессы не обращаются непосредственно к физической памяти. Вместо этого ядро связывает каждый процесс с собственным уникальным виртуальным адресным пространством данного процесса. Это адресное пространство является линейным. Его адреса начинаются с нуля, непрерывно увеличиваясь вплоть до заданного максимального значения. Кроме того, адресное пространство является плоским; оно существует в единой области, непосредственно доступной и не требующей сегментирования.

Страницы и их подкачка

Память состоит из битов, причем восемь бит (как правило) составляют один байт. Байты складываются в слова, а слова - в страницы. При управлении памятью наиболее важной из этих концепций является именно страница. Страница — это наименьшая адресуемая сущность в памяти, которой может управлять блок управления памятью (MMU), поэтому можно сказать, что страницы «нарезаются» из адресного пространства процесса. Размер страницы зависит от применяемой машинной архитектуры. Наиболее распространенными размерами являются 4 Кбайт

(в 32-битных системах) и 8 Кбайт (в 64-битных системах)1. В 32-битном адресном пространстве содержится около миллиона страниц размером по 4 Кбайт каждая. В 64-битном адресном пространстве на несколько порядков больше страниц по 8 Кбайт. Процесс не обязательно будет обращаться ко всем этим страницам; конкретная страница может ничему не соответствовать.

Совместное использование и копирование при записи.

Разделяемые данные могут быть доступны только для чтения, записи или предоставляться для одного и другого. Когда процесс записывает информацию на совместно используемую страницу, допускающую такую запись, может произойти одна из двух вещей. В простом случае ядро разрешает запись, после чего все процессы, использующие эту страницу, могут видеть результат записи.

Как правило, если мы разрешаем множественным процессам считывать одну и ту же страницу или записывать на нее информацию, требуется обеспечить некоторую степень координации и синхронизации между этими процессами, но на уровне ядра запись «просто работает» и все процессы, разделяющие данные, немедленно видят все происходящие изменения.

Во втором случае блок управления памятью может перехватить операцию записи и выдать исключение. В ответ ядро прозрачно создает новую копию страницы для записывающего процесса и позволяет процессу продолжать запись информации уже на новой странице. Такой подход называется копированием при записи (COW). Фактически процессы получают к разделяемым данным доступ для чтения, благодаря чему экономится место. Однако если процесс пытается записать информацию на разделяемую страницу, то на лету получает уникальную копию этой страницы.

Таким образом, ядро всегда может работать с учетом наличия собственной копии страницы у каждого записывающего процесса. Копирование при записи происходит постранично, поэтому такая техника фактически позволяет разделять огромный файл между многими процессами и отдельные процессы будут получать уникальные физические копии только страниц, на которые они сами записывают информацию.

Динамическая память

Выделение и освобождение памяти

Память также предоставляется в форме автоматических и статических переменных, но основа любой системы управления памятью — это выделение, использование и, наконец, возврат динамической памяти. Динамическая память возвращается во время выполнения, а не во время компиляции, причем в размере, который может быть неизвестен вплоть до момента выделения. Разработчик прибегает к использованию динамической памяти, когда объем требуемой памяти или длительность ее использования может варьироваться, причем точные значения размера и длительности становятся известны только после запуска программы. Например, вы хотите сохранить в памяти содержимое файла или пользовательского ввода, полученного через клавиатуру. Размер файла точно не известен, а пользователь может ввести достаточно много информации, поэтому размер буфера будет варьироваться и вам потребуется его динамически увеличивать по мере считывания все большего количества данных.

При создании массива с фиксированными размерами под него выделяется определенная память. Например, пусть у нас будет массив с пятью элементами:

```
double numbers[5] = {1.0, 2.0, 3.0, 4.0, 5.0};
```

Для такого массива выделяется память 5 * 8 (размер типа double) = 40 байт. Таким образом, мы точно знаем, сколько в массиве элементов и сколько он занимает памяти. Однако это не всегда удобно. Иногда бывает необходимо, чтобы количество элементов и соответственно размер выделяемой памяти для массива определялись динамически в зависимости от некоторых условий.

Например, пользователь сам может вводить размер массива. И в этом случае для создания массива мы можем использовать динамическое выделение памяти.

Для управления динамическим выделением памяти используется ряд функций, которые определены в заголовочном файле **stdlib.h**:

- **malloc()**. Имеет прототип

```
1  void *malloc(unsigned s);
```

Выделяет память длиной в s байт и возвращает указатель на начало выделенной памяти. В случае неудачного выполнения возвращает **NULL**

- **calloc()**. Имеет прототип

```
1  void *calloc(unsigned n, unsigned m);
```

Выделяет память для n элементов по m байт каждый и возвращает указатель на начало выделенной памяти. В случае неудачного выполнения возвращает **NULL**

- **realloc()**. Имеет прототип

```
1 void *realloc(void *bl, unsigned ns);
```

Изменяет размер ранее выделенного блока памяти, на начало которого указывает указатель bl, до размера в ns байт. Если указатель bl имеет значение **NULL**, то есть память не выделялась, то действие функции аналогично действию **malloc**

- **free()**. Имеет прототип

```
1 void *free(void *bl);
```

Освобождает ранее выделенный блок памяти, на начало которого указывает указатель bl.

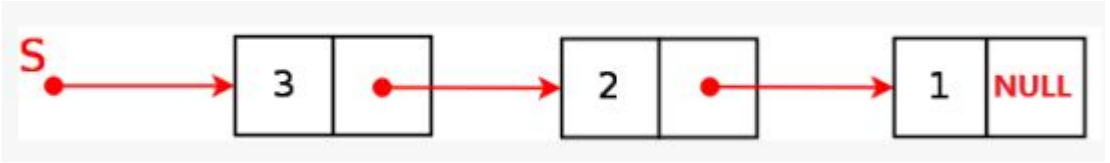
Динамические структуры данных

Допустим, мы пишем программу, позволяющую вводить данные на сотрудников организации. Количество сотрудников неизвестно. Можно было бы создать массив записей с запасом. Однако, если данных о каждом сотруднике много, то каждая запись занимает много памяти; получается, что мы будем расходовать много памяти в пустую, если сотрудников мало.

1. В программе определяем структурный тип данных с "кое-какой изюминкой" и создаем переменную указатель на него. В итоге при запуске программы память выделяется только под указатель.
2. В процессе выполнения программы, в случае возникновения необходимости в создании структуры, с помощью специальной функции выделяем память под хранение данных (полей структуры).
3. Присваиваем указателю адрес, по которому расположена только что созданная структура.
4. Когда поступает команда на создание следующей структуры, снова с помощью функции выделяется память, а указателю присваивается адрес этой новой структуры.
5. "Изюминка" определенного ранее структурного типа данных заключается в том, что одним из его полей является указатель на структуру этого же типа.
6. В это поле-указатель записывается адрес на структуру, которая была создана перед данной структурой.

Стек

Таким образом получается цепочка взаимосвязанных структур. Самая первая созданная структура не имеет ссылки на другую структуру. Ее поле-указатель имеет значение NULL. Вторая созданная структура ссылается на первую, третья на вторую и т.д. Адрес последней созданной структуры хранится в переменной-указателе, которая была объявлена в программе программистом.



Создание стека

```c
#include <stdio.h>
#include <stdlib.h>

struct stack {
    int data;
    struct stack *next;
};

/* присоединение элемента к голове,
возврат адреса головы */
struct stack *create(struct stack *, int);

// просмотр стека
void list(struct stack *);
```

```c
int main() {
    int i, n;
    // адрес, указывающий на голову стека
    struct stack *head;
    head = NULL;

    scanf("%d", &n);
    for (i=0; i <= n; i+=5) {
       head = create(head,i);
       printf("%d<--", head->data);
    }
    printf("\n");
    list(head);
    free(head);
}
```

```
struct stack *create(struct stack *head,
                     int x) {

    // указатель на новую структуру
    struct stack *element;

     // выделяем память
    element = (struct stack *)malloc(
                  sizeof(struct stack));
    element->next = head;
    element->data = x;
    return element;
}
```

```c
void list(struct stack *p){
    // пока не конец стека
    while (p != NULL) {
      printf("%d-->", p->data);
      // продвижение по списку
      p = p->next;
    }
    printf("\n");
}
```

Ключевые термины

Адрес сегмента – это одно из машинных слов, составляющих адрес динамического элемента, которое представляет собой адрес первого элемента структуры.

Адресное поле (поле связок) – это поле структуры, в котором содержится указатель, связывающий данный элемент с другими элементами структуры.

Динамические структуры данных – это структуры данных, память под которые выделяется и освобождается не на этапе компиляции, а в процессе работы программы.

Динамический элемент – это элемент динамической структуры, который в конкретный момент выполнения программы может либо существовать, либо отсутствовать в памяти.

Динамическое распределение памяти – это выделение памяти под отдельные элементы в тот момент, когда они "начинают существовать" в процессе выполнения программы.

Информационное поле (поле данных) – это поле структуры, в котором содержатся непосредственно обрабатываемые данные.

Связное представление данных – это установление связи между элементами динамической структуры через указатели.

Смещение – это одно из машинных слов, составляющих адрес динамического элемента, которое представляет собой изменение адреса относительно первого элемента структуры.

Выводы

1. В программах возникает необходимость обрабатывать данные, размер которых заранее неизвестен.
2. Для данных с достаточно большим или переменным размером используются динамические структуры.
3. Динамические структуры не имеют имени, под них выделяется память в процессе выполнения программы, количество их элементов может не фиксироваться, в процессе выполнения программы может меняться характер взаимосвязи между элементами структуры.
4. Каждой динамической структуре ставится в соответствие статическая переменная – ее адрес.
5. Представление динамических структур в памяти определяется как связное.
6. Связное представление данных в программах имеет как достоинства, так и недостатки.
7. Существует классификация динамических структур данных в зависимости от связей между элементами и допустимых операций.
8. Элемент динамической структуры состоит как минимум из двух полей: адресного и информационного.
9. Адресное поле формируется из двух слов: адрес сегмента и смещение.
10. Доступ к данным в динамических структурах осуществляется с помощью операции косвенного выбора.